

PasteCraft Architecture Pack

Combined from:

1. docs/architecture/FORWARD-ARCHITECTURE.md
2. docs/architecture/engineering-readiness-roadmap.md
3. ArchiGuide/architecture-deep-reference.md

Generated for offline download.

Forward Architecture — Modular Vertical Slices + Legacy Facades

Cutoff: 2026-07-05. Code written or materially changed **after** this date follows this doc. Shippable legacy stays until touched.

Named pattern: Modular Vertical Slices + Legacy Facades (Arkitect: modular-monolith + clean-slice-fusion remix + strangler-fig migration + ACL at legacy seams).

1. Cutoff principle

Rule	Detail
Ship first	No big-bang rewrite. Extension stays shippable.
New work	Vertical slices + thin entry wiring only.
Legacy	Untouched files stay put. Migrate only when you edit that area.
Bridges	New code talks to legacy through one facade per domain — never scattered legacy imports.
Gates	CodeScene on changed files; Ezequiel SUCCESS before SuccessLog / task-branch commit.

2. Legacy vs new — ONE identification approach

Primary signal: path convention. Secondary: file header marker.

Forward (new) paths — allowed for new logic

Layer	Path pattern
Content slice	extension/content/<feature>/
Popup slice	extension/popup/features/<feature>/
Background slice	extension/background/handlers/<cluster>/ , extension/background/messaging/
Supabase slice	extension/supabase/<module>.js
Shared kernel	extension/shared/ (constants, messaging, IDs — no feature logic)
Legacy facades	extension/bridges/<domain>/

Legacy (pre-cutoff) — do not add business logic here

Path	Role
extension/popup.js	Popup shell (~1,705 lines)
extension/content/quick-paste/ quick-paste.js	QuickPaste monolith (~2,500 lines)
Root shims	extension/background.js , extension/supabase-client.js , extension/content-script.js
Ad-hoc roots	extension/tiered-storage.js , extension/indexeddb- store.js , etc. until facaded

Do not create `extension/legacy/` or mass-move files. Legacy = **registry + path outside forward patterns**.

Secondary marker (optional, on touch)

First line block when editing any file:

```
/** @legacy-cutoff pre-2026-07-05 – migrate via extension/bridges/<domain>/ */  
/** @forward-slice clips – canonical slice code */
```

Agents: if path is ambiguous, check marker; if neither, treat as legacy until facaded.

3. Target architecture (Arkitekt-aligned)

Arkitekt ID	Role in PasteCraft
modular-monolith	Single MV3 deployable; strict internal module boundaries
vertical-slice	Feature folders own UI + orchestration in their context
strangler-fig	Replace legacy incrementally when touched
anti-corruption-layer	extension/bridges/ * translates legacy shapes ↔ slice contracts
clean-slice-fusion	Thin domain-facing APIs inside slices; adapters at edges

Patterns (use, don't over-build): Strategy (site adapters, sync providers), Facade + Adapter (bridges), Command + Mediator (background router, queues), Decorator (wrap legacy calls during migration).

Deferred: microservices, CQRS, event sourcing, DDD aggregates, abstract-factory ceremony.

4. Folder map — NEW work only

```
extension/
├── bridges/                                # NEW — legacy facades only (ACL)
│   ├── clips/
│   │   └── clips.legacy-facade.js
│   ├── sync/
│   │   └── sync.legacy-facade.js
│   └── quick-paste/
│       └── qp.legacy-facade.js
├── content/
│   └── <feature>/                        # controller, constants, events, styles, adapters/
├── popup/
│   └── features/<feature>/                # controller, service, state, render, events
├── background/
│   ├── messaging/                        # message-types.js, router.js
│   └── handlers/<cluster>/                # one cluster per message family
├── supabase/                             # one concern per module; thin supabase-client.js shim
└── shared/                               # storage keys, clip-id, messaging — no feature rules
```

Context boundaries (MV3):

Context	May import	Must NOT
Content script	shared/ , own slice, bridges/	Popup modules, direct Supabase
Popup	shared/ , own slice, bridges/ , supabase/ via service	Content slices
Background	shared/ , handlers, supabase/	DOM, content UI
Bridge	Legacy modules + shared/	Sibling bridge internals

5. Bridge / facade rules

1. **One public module per domain** under `extension/bridges/<domain>/`.
2. **Slices import bridges — not legacy.** Example: `clips.service.js` → `bridges/clips/clips.legacy-facade.js` → `popup.js` / `tiered-storage.js` / `supabase/sync-clips.js`.
3. **Bridge owns translation:** legacy float IDs, tombstones, sync queue side effects, chrome.storage key quirks.
4. **No re-export sprawl:** facades export a small named API (≤10 functions per domain).
5. **Shrink over time:** as legacy is extracted into slices, facade delegates to slice code; delete legacy path last.
6. **Messages cross context:** use `message-types.js` + router — not ad-hoc `chrome.runtime.sendMessage` strings in slices.

Facade sketch (doc only):

```
// extension/bridges/clips/clips.legacy-facade.js
export async function updateClipTitle(clipId, title) { /* wrap legacy + sync queue */ }
export async function deleteClip(clipId) { /* tombstone + sync; single exit */ }
```

6. Strangler migration — when you touch legacy

```
flowchart LR
    subgraph New["Forward slices"]
        S[popup/features/clips]
        B[bridges/clips]
    end
    subgraph Legacy["Pre-cutoff legacy"]
        P[popup.js]
        T[tiered-storage.js]
        Y[supabase/sync-clips.js]
    end
    S --> B
    B --> P
    B --> T
    B --> Y
```

Step	Action
1	Assess — Arkitect diagnose_repository + analyze_refactoring_opportunities for the slice
2	Facade first — new behavior calls legacy through bridges/<domain>/
3	Extract — Move Method / Extract Class into slice folder (Arkitect: moving-features)
4	Rewire — Entry shell + facade point at slice; legacy body shrinks
5	Verify — Smoke test + CodeScene; no storage key renames without migration
6	Retire — Remove legacy path only when facade has zero legacy imports

Extract-on-touch: editing popup.js clip handlers → extract to popup/features/clips/ or facade first, never grow the monolith.

7. What NOT to do

- Cross-slice imports (merchant/ ↔ widget/).
- New business logic in popup.js , quick-paste.js , or root monoliths.
- Direct legacy imports from slice files (bypass bridge).
- Mass rename / extension/legacy/ dump without migration plan.
- Storage key renames without SCHEMA_VERSION migration.
- Auto-refactor spaghetti without explicit migration intent (Arkitect policy).
- Singleton service layers for convenience.

8. First bridge targets (when touched)

Priority	Slice	Why	Facade domain
1	Clips CRUD	Title update / delete bugs trace to scattered sync + legacy IDs	bridges/clips/
2	Widget	Phase 0 slice done — pattern reference for content	Already forward; extend, don't regress
3	Merchant	Adapters + queues already partial strangler	merchant.adapters/ stays; add bridges/merchant/ only if popup/sync legacy leaks in

Clips title update / delete → facade pattern

Symptom	Strangler fix
Title save hits legacy path + sync race	updateClipTitle() in facade: normalize ID (shared/clip-id.js), single write path, queue one sync op
Delete resurrects on sync	deleteClip() in facade: tombstone locally, enqueue delete, block legacy merge resurrect
Popup + QuickPaste diverge	Both call same facade API — not separate legacy copies

9. Arkitect MCP — recommendations captured

Diagnosis (2026-07-05):

- **Selected architecture:** modular-monolith (score 1.0) — single deployable, enforced internal boundaries.
- **Remix:** clean-slice-fusion — Vertical Slice + Clean Architecture dependency direction.
- **Migration:** strangler-fig + anti-corruption-layer at legacy seams.
- **Intent:** migration / guide-new-foundation — **auto-refactor: false**.
- **Patterns:** Strategy, Decorator, Composite, Command, Mediator; Facade + Adapter at bridges.
- **Smells:** duplicate code, god class (popup.js , quick-paste.js).
- **Techniques:** Move Method, Extract Class when splitting god classes.
- **Tags:** vertical-slice-delivery , legacy-strangler , plugin-system , monolith-foundation .

Relation to existing docs:

Doc	Role
This file	Canonical cutoff + bridge strategy
vertical-slice-modularity.mdc	Implementation detail for slice layout

Doc	Role
refactor-plan-composer-first.md	Phased execution roadmap (MV3-B, Supabase 2.x, QuickPaste 3.x)
architecture-reference.mdc	MV3 data flow + component responsibilities

10. Re-run Arkitect for new features

Per feature slice (before coding):

```
# MCP intake – paste into agent or apply_workbench_intake
repoPath: c:\Dev\PasteCraft
requestedOutcome: "Implement [FEATURE] ONLY in forward paths. Legacy via extension/
bridges/[domain]/. DO NOT edit: [forbidden paths]."
explicitRefactorIntent: true
category: moving-features
architectureId: vertical-slice
remixId: clean-slice-fusion
requirementTags: [vertical-slice-delivery, legacy-strangler, mv3-extension]
```

Tools (in order):

1. `diagnose_repository` — confirm architecture path
2. `recommend_patterns` — pattern shortlist for the feature
3. `analyze_refactoring_opportunities` — if touching legacy registry files

Arkitekt Desktop (optional): `apply_workbench_intake` with `autoRun.diagnosis: true` when Desktop bridge is running — saves preset `pastecraft-forward-<feature>`.

Manual each feature? Yes for structural work (new slice, legacy touch, bridge). **No** for trivial bugfix wholly inside an existing forward slice file.

11. Agent checklist (every implementation)

1. Confirm date > cutoff → forward paths only.
2. Pick slice folder OR bridge domain before coding.
3. Legacy needed? → add/extend `extension/bridges/<domain>/` — no direct legacy import from slice.
4. Register message types in `message-types.js` if cross-context.
5. CodeScene review on touched files.
6. Ask Ezequiel: *Is the implementation successful?* before `SuccessLog`.

Engineering Readiness Roadmap

Collaborative doc for Ezequiel + agent. **No implementation, deployment, key rotation, or SuccessLog from this file alone.**

Separate commit scope: Local Stripe webhook / Basil hardening changes — review, test, and commit on their own branch. Do not mix with this roadmap work.

Completed baseline

Item	Status	Notes
Chrome in-extension privacy disclosure	Done	extension/popup/features/privacy/ — Settings Privacy & Data
Website privacy policy	Done	website/privacy.html
PostgreSQL version	Done	Production PG 17.6 — no upgrade needed
Stripe Basil period handling	Deployed	stripe-webhook v32; API version pinned 2023-10-16 intentionally
MV3 compliance	Done	Module SW + content; no inline handlers
TypeScript 5 floor	N/A	Mostly vanilla JS
Node 22 CI/deployment	Done	Already aligned
Forward Architecture cutoff	Canonical	docs/architecture/FORWARD-ARCHITECTURE.md (2026-07-05)

Priority ladder

#	When	Workstream
1	Now	Edge Partner Center privacy answers + Chrome/Edge listing alignment
2	Before next store upload	Permissions, hosts, privacy URL, auth/sync/AI/billing copy match reality
3	Before Stripe API-version bump	Review deployed webhook separately; billing smoke tests
4	Next engineering phase	Supabase publishable/secret-key + ES256 — research first (Phase A)
5	Before OAuth maintenance	Record real Chrome + Edge extension IDs
6	Optional	supabase-js upgrade (^2.102.1 today)
7	Ongoing	Forward Architecture slice extraction — only when touching legacy

Workstream summary

Workstream	Integrated	Missing	To do
Supabase API keys / ES256	Legacy <code>anonKey</code> works; Edge Functions use service role server-side	No <code>sb_publishable_</code> / <code>sb_secret_</code> ; no ES256 migration plan on disk	Phase A inventory → phased B–G (see below)
Edge Partner Center Privacy	<code>PARTNER_CENTER_INFO.md</code> , <code>website/privacy.html</code> , in-app disclosure	Form answers not drafted; manifest in doc is stale (3.0.6 vs 3.0.19)	Agent drafts answers; Ezequiel submits in Partner Center
Store listing alignment	Docs under <code>docs/publishing/</code> , <code>edge-store-assets/</code>	Chrome dashboard copy manual; Edge doc permissions outdated	Reconcile manifest ↔ listing ↔ privacy before upload
OAuth dual-ID docs	<code>chrome.identity.getRedirectURL()</code> in code	TBD_CHROME_ID / TBD_EDGE_ID in publishing rules	Ezequiel supplies IDs → agent updates docs + allowlist checklist
supabase-js upgrade	<code>^2.102.1</code> in <code>package.json</code> ; Node 22	Not on latest 2.110+	Optional maintenance window + auth/sync tests
Forward Architecture	Cutoff doc + slice patterns + refactor progress	Legacy monoliths remain (<code>popup.js</code> , <code>quick-paste.js</code> , etc.)	Extract on touch per <code>REFACTOR_REMAINING.md</code>
Stripe webhook (local)	Deployed v32 in prod	Uncommitted local changes possible	Separate branch/commit — not part of this roadmap

1. Supabase API keys + ES256 migration

What it means

Key type	Where	Rule
Legacy <code>anon</code> JWT	Extension + Edge today	Current production path
<code>sb_publishable_...</code>	Extension, website client	Public <code>apikey</code> header only
<code>sb_secret_...</code>	Edge Functions, server only	Never in extension, git, or store zip
User access JWT	Authorization: Bearer <token>	Real user token — not publishable/secret key

Current status: Deferred intentionally. Only legacy anon key verified. No safe replacement to flip to yet.

When

Start research when	Do not start before
Supabase migration deadline announced	Store submission
Security policy requires ES256	Billing change
Major auth/infrastructure release planned	Large refactor or launch week
Maintenance window + login/sync test capacity	Period without manual auth testing

Prerequisites

Ezequiel	Agent
Confirm Supabase project + prod env	Inventory every key/header reference
Create/enable new key types in Dashboard	Classify client vs user JWT vs backend secret
Keep legacy key active during window	Update client config + Edge env var names
List Edge Functions using <code>SUPABASE_ANON_KEY</code> / <code>SUPABASE_SERVICE_ROLE_KEY</code>	Header helpers, tests, staged deploy checklist
Access Auth, API, Edge secrets, logs	RLS + auth path review
Approve maintenance window + rollback owner	Cannot create or rotate prod keys without explicit approval

Key touchpoints (inventory starting list)

- `extension/config.example.js`
- `extension/supabase/auth.js`, `ai-functions.js`, `ai-history-sync.js`, `subscription.js`
- `extension/popup/features/auth/auth.service.js`, `billing/billing.service.js`
- `extension/background/handlers/messages-internal.js`
- `supabase/functions/*` (checkout, portal, webhook, admin, redeem, usage, announcements, `_shared/ai_workflow.ts`)

Phased plan (A–G)

Phase A — Read-only inventory (*research only*)

- [] Every `anonKey` / `SUPABASE_ANON_KEY` / service-role reference
- [] Every `createClient()`, `apikey`, `Authorization` header
- [] Every Edge Function auth model + `verify_jwt` setting
- [] Classify: `apikey: sb_publishable_...` + `Authorization: Bearer <user JWT>` — never Bearer with publishable/secret key

Success: Written inventory doc, zero code changes.

Rollback: N/A.

Phase B — Dual-key window

- Ezequiel creates new keys; legacy stays active
- Record: creation date, env mapping, rollback key, planned revocation date

Success: Both key types exist; legacy not revoked.

Rollback: Continue on legacy only.

Phase C — Backend preparation

Move server-only functions to `sb_secret_...` after compatibility check.

Likely functions: `create-checkout`, `create-portal-session`, `stripe-webhook`, `admin-api`, `admin-alerts`, `redeem-coupon`, `usage-beacon`, `get-announcements`, `_shared/ai_workflow.ts`.

Success: Backend uses secret key server-side only.

Rollback: Restore prior Edge env vars.

Phase D — JWT validation preparation

- Map validation: Edge gateway vs function vs `supabase-js` vs RLS
- Do not blindly set `verify_jwt = false`
- Stripe webhook: gateway JWT off OK if Stripe signature verified
- User-facing functions: retain JWT validation; use documented ES256/JWKS approach if manual

Success: Auth model documented per function.

Rollback: Revert `config.toml` / function auth settings.

Phase E — Client migration

Extension uses publishable key for `apikey`; user tokens in `Authorization`.

Update: client init, REST headers, auth refresh, function invoke, checkout messages, config examples.

Success: No secret key in bundle; headers correct.

Rollback: Restore prior `config` / client key.

Phase F — Staging verification

Test: email + Google login, refresh, sign-out, session restore, RLS clips/categories/notes/settings, Realtime, AI, subscription, checkout, webhook, admin functions.

Success: All paths pass on staging with new keys.

Rollback: Revert client + backend to legacy keys.

Phase G — Production flip

1. Deploy backend env → deploy extension → monitor auth/errors
2. Keep legacy key active through activity window
3. Re-check login, sync, RLS, billing
4. Revoke legacy key only after rollback window closes

Success criteria (all phases):

- [] No secret key in extension or website client
- [] Client: `apikey` = publishable; `Authorization` = user JWT
- [] Login, refresh, RLS, sync, billing unchanged for existing users
- [] Stripe webhooks still signature-validated

Rollback (any phase after B): Restore prior client key + Edge env vars; keep new keys for investigation; **no** DB reset or emergency RLS changes.

Forward Architecture boundary

Treat as **controlled infrastructure slice** — not a full Supabase refactor.

Layer	Path
Supabase client modules	<code>extension/supabase/</code>
Shared header/config contracts	<code>extension/shared/</code>
Cross-context requests	<code>extension/background/handlers/</code>
Backend auth helpers	<code>supabase/functions/_shared/</code>
Forbidden	New logic in legacy monoliths

2. Edge Partner Center Privacy

What it is

Manual privacy/disclosure form in Microsoft Partner Center. Code cannot submit it.

Reference	Path
Submission notes	<code>edge-store-assets/PARTNER_CENTER_INFO.md</code>
Live policy	<code>https://pastecraft.com/privacy</code> → <code>website/privacy.html</code>
In-app disclosure	<code>extension/popup/features/privacy/</code>

When

Before next Edge publication. **Independent** of Supabase key migration — describe **current** behavior.

Prerequisites

Ezequiel	Agent
Partner Center access	Draft form answers from manifest + privacy.html + Settings copy
Intended listing version	Flag contradictions (e.g. stale permissions in <code>PARTNER_CENTER_INFO.md</code>)
Confirm privacy URL live	Align Chrome ↔ Edge wording
Confirm active telemetry/analytics	Remove unsupported claims

Success criteria

- ☐ Remote code: **No**
- ☐ Data: account info, user-saved content, settings, support messages (where used)
- ☐ Cloud: Supabase; Billing: Stripe; AI: only on user invocation
- ☐ Privacy URL live, no placeholders
- ☐ Matches Chrome listing disclosures

Rollback / testing

- ☐ Open privacy URL in clean browser
- ☐ Every stated permission exists in `extension/manifest.json`
- ☐ Every claimed feature exists in shipped build
- If rejected: fix listing/disclosure text — do not change product behavior to match inaccurate copy

3. Chrome + Edge store listing alignment

When

Before any Chrome or Edge upload; whenever permissions, hosts, auth, sync, AI, or billing change.

Prerequisites

Ezequiel	Agent
Review store dashboard fields	Compare listing ↔ <code>manifest.json</code>
Confirm audience + disclosures	Remove stale version claims (e.g. "v1.0" / "v2.0 coming")
Submit listing updates	Permission justifications; flag dev-only hosts

Current manifest snapshot (verify before upload)

Area	Current (extension/manifest.json)
Version	3.0.19
Permissions	contextMenus , storage , activeTab , tabs , scripting , identity , clipboardWrite
Hosts	<all_urls> , *.supabase.co , *.blob.core.windows.net , accounts.google.com , pastecraft.com
CSP connect-src	Supabase, blob, Google, pastecraft.com

PARTNER_CENTER_INFO.md still lists older permissions (e.g. api.openai.com , 127.0.0.1:7242)
— reconcile before submit.

Success criteria

- [] Store copy = actual features
- [] Permissions justified truthfully
- [] Privacy + terms URLs work
- [] Chrome and Edge describe the **same** extension/ zip
- [] Screenshots/promo assets current (edge-store-assets/)

Rollback / testing

- Keep last accepted listing text
- Manifest change → full Section G checklist (.cursor/rules/production-publishing-safety.mdc)

4. Optional supabase-js upgrade

Current	@supabase/supabase-js ^2.102.1 (package.json)
When	Planned dependency window — not bundled with key migration unless required
Ezequiel	Approve target version + window
Agent	Upgrade lockfile, run tests, check deprecated auth options
Test	Session restore, refresh, Realtime, sync, Edge Function calls
Rollback	Restore prior package + lockfile; re-run auth/sync tests
Rule	Do not combine with key rotation or large refactor

5. OAuth dual-ID documentation

Code	<code>chrome.identity.getRedirectURL()</code> — correct portable pattern
Docs	TBD_CHROME_ID / TBD_EDGE_ID in <code>production-publishing-safety.mdc</code>
When	Before prod OAuth testing, store submit with login, Supabase redirect review
Ezequiel	Copy real IDs from Chrome + Edge dashboards; confirm Google Cloud + Supabase Auth allowlists
Agent	Replace placeholders; parity checklist; verify no hard-coded ID replaced runtime redirect
Success	Both IDs in Google OAuth + Supabase redirects + publishing docs; login works Chrome + Edge (fresh + existing session)
Rollback	Add new redirects before removing old; compare generated redirect URL to allowlist exactly

6. Forward Architecture connection

Readiness work **does not** replace the refactor roadmap.

Doc	Role
<code>FORWARD-ARCHITECTURE.md</code>	Canonical cutoff + bridges
<code>master-refactor-roadmap.md</code>	Popup/monolith phased execution
<code>REFACTOR_REMAINING.md</code>	Daily automation queue

Order of operations:

1. Urgent compliance + production readiness (this doc)
2. Keep extension shippable
3. On legacy touch → one vertical slice + bridge if needed
4. Arkitect before structural work; CodeScene on changed files
5. Stable storage keys + auth contracts unless explicit migration planned

Discussion — choose next position

Review this file, then pick **A**, **B**, or **C**.

A — Approve / use this roadmap only

Integrated	Baseline items done; priority ladder; workstream tables; phased Supabase plan; Forward Architecture link
Missing	Formal SUCCESS approval; <code>implementations.md</code> entry (only after Ezequiel confirms)

To do	Ezequiel reads + adjusts priorities; agent updates doc if gaps found; then pick B or C
Agent next	Hold implementation until position chosen
Ezequiel next	Confirm ladder order or reprioritize

B — Prepare Edge Partner Center privacy answers next

Integrated	Privacy slice, website policy, <code>PARTNER_CENTER_INFO.md</code> shell
Missing	Drafted Partner Center form answers; manifest ↔ doc reconciliation; Chrome parity pass
To do	Agent drafts Q&A from manifest <code>3.0.19</code> + <code>privacy.html</code> + Settings disclosure; Ezequiel reviews + submits in Partner Center
Prerequisites	Partner Center login; confirmed privacy URL
Success	Submitted form matches shipped extension; no placeholder text
Not in scope	Supabase key migration; Stripe webhook commit

C — Start research-only Phase A (Supabase API keys)

Integrated	Known touchpoint list (above); deferred status documented
Missing	Full grep inventory; per-function <code>verify_jwt</code> matrix; header classification table
To do	Agent runs read-only inventory → deliver Phase A report; zero code/key/dashboard changes
Prerequisites	Ezequiel confirms target Supabase project; approves research-only scope
Success	Complete inventory artifact; go/no-go for Phase B
Not in scope	Key creation, rotation, deploy, extension config changes

Quick checklists

Before next store upload

- ☐ `manifest.json` version > last published
- ☐ No `"key"` field in package zip
- ☐ Privacy URL live
- ☐ Partner Center / Chrome dashboard copy matches manifest
- ☐ Section G smoke test if manifest touched

Before Supabase key migration (any phase after A)

- ☐ Phase A inventory approved

- ☐ Maintenance window scheduled
- ☐ Rollback owner named
- ☐ Not same week as store submit or Stripe API bump

Before OAuth doc update

- ☐ Real Chrome ID from dashboard
 - ☐ Real Edge ID from dashboard
 - ☐ Supabase Auth redirect allowlist checked
-

Related docs

- [FORWARD-ARCHITECTURE.md](#)
- [master-refactor-roadmap.md](#)
- [REFACTOR_REMAINING.md](#)
- [PARTNER_CENTER_INFO.md](#)
- [production-publishing-safety.mdc](#)

Architecture Deep Reference

Full reference for the browser extension + Supabase backend system.

References:

- Chrome Extension MV3: <https://developer.chrome.com/docs/extensions/mv3/intro/>
- Supabase Architecture: <https://supabase.com/docs/guides/functions/architecture>
- Shadow DOM (MDN): https://developer.mozilla.org/en-US/docs/Web/API/Web_components/Using_shadow_DOM
- Edge (Chromium): <https://learn.microsoft.com/en-us/microsoft-edge/extensions-chromium/>

Full Directory Map

```
my-extension/
├── manifest.json
├── background/
│   ├── service-worker.js          # Entry; delegates to handlers
│   └── handlers/
│       ├── auth.handler.js
│       ├── checkout.handler.js
│       └── feature.handler.js
├── content/
│   ├── content.js                # Entry; imports feature modules
│   ├── widget.css                # Injected inside shadow root
│   └── modules/
│       ├── ui-injector.js        # Shadow DOM factory
│       ├── messaging.js          # sendToBackground helper
│       └── feature-a.js          # One file per feature
├── popup/
│   ├── popup.html
│   ├── popup.js
│   └── popup.css
├── options/
│   ├── options.html
│   ├── options.js
│   └── options.css
├── shared/
│   ├── api.js                   # Supabase client
│   ├── auth.js                  # Session helpers
│   ├── store.js                 # Observer state
│   ├── messaging.js             # Shared message util
│   ├── storage-adapter.js        # chrome.storage adapter for Supabase
│   └── constants.js             # STORAGE_KEYS, MESSAGE_TYPES, etc.
├── assets/
│   ├── icons/
│   └── styles/
│       └── tokens.css            # CSS custom properties only
└── supabase/
    └── migrations/
```

```

├── 20260331120000_create_profiles.sql
├── 20260331130000_create_entries.sql
├── functions/
│   ├── create-checkout/
│   │   └── index.ts                # Stripe checkout session creator
│   ├── stripe-webhook/
│   │   └── index.ts                # Webhook receiver + DB updater
│   └── check-subscription/
│       └── index.ts                # Auth-gated status check
├── .claude/
│   └── skills/
│       ├── id-management/SKILL.md
│       ├── css-isolation/SKILL.md
│       ├── background-service/SKILL.md
│       ├── content-script/SKILL.md
│       ├── popup-ui/SKILL.md
│       ├── supabase-backend/SKILL.md
│       ├── stripe-payments/SKILL.md
│       ├── db-migrations/SKILL.md
│       └── core-conventions/SKILL.md
├── docs/agent-guides/
│   ├── architecture.md            # this file
│   └── data-flow.md               # full end-to-end flows

```

Component Responsibilities

Component	Responsibility	Must NOT
service-worker.js	Route messages, open tabs, trigger auth	Store state in memory
content.js	Init feature modules, check auth, inject UI	Call Supabase directly
content/modules/	Feature logic on host pages	Contain inline styles or id generation
popup.js	UI state, user actions	Contain business logic
shared/api.js	Single Supabase client	Be imported from content scripts directly
shared/auth.js	Session read/write	Generate or store userId separately
Edge Functions	Server-side logic, Stripe, DB admin writes	Be called with secret keys from the extension

Context Isolation (Important for Content Scripts)

Content scripts run in an isolated world -- they cannot access the host page's JS variables, and the host page cannot access the extension's content script variables.

The extension has 3 separate JS contexts:

1. Service worker (background) -- has full chrome. *API access*
2. *Popup / options pages* -- have full chrome. *API access*
3. Content scripts -- limited chrome.* API (mainly chrome.runtime, chrome.storage.local)

Content scripts cannot call chrome.tabs, chrome.windows, or chrome.identity.

For those operations, they must send a message to the service worker.

Reference: <https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts>

Data Flow: User Creates an Entry

```
1. User types in content script widget (Shadow DOM)
2. content/modules/feature-a.js collects form data
3. sendToBackground({ type: 'feature:create', title, body })
4. service-worker.js routes to feature.handler.js
5. feature.handler.js calls shared/auth.js -> getUserId() -> session.user.id
6. feature.handler.js calls supabase.from('entries').insert({ title, body, user_id:
  userId })
  -- id is OMITTED -- Postgres assigns gen_random_uuid()
7. Supabase returns { id: 'server-uuid', title, body, user_id, created_at }
8. sendResponse({ ok: true, entry: data })
9. content script renders the new entry using data.id from the response
```

Data Flow: Stripe Payment

```
1. User clicks Upgrade in popup
2. popup.js sends message { type: 'checkout', priceId, userEmail }
3. checkout.handler.js POSTs to Supabase Edge Function: create-checkout
  -- Authorization: Bearer {supabase_anon_key} header included
  -- userId NOT in body -- Edge Function extracts from JWT
4. Edge Function verifies JWT, calls stripe.checkout.sessions.create()
  -- metadata: { supabase_user_id: user.id } stored on Stripe session
5. Edge Function returns { checkout_url }
6. Background opens chrome.tabs.create({ url: checkout_url })
7. User completes payment on Stripe hosted page
8. Stripe POSTs to stripe-webhook Edge Function
9. Webhook verifies Stripe-Signature header
10. Webhook reads userId from event.data.object.metadata.supabase_user_id
11. Webhook updates profiles.subscription_status = 'active'
12. Extension checks subscription status on next popup open via check-subscription
```

Browser Compatibility

All Chromium browsers use the same MV3 API:

- Chrome: <https://developer.chrome.com/docs/extensions/>
- Edge: fully supports MV3 (enforced from July 2022)

Reference: <https://blogs.windows.com/msedgedev/2020/10/14/extension-manifest-chromium-edge/>

- Brave, Arc, Opera: same Chromium extension API
- Use `chrome. namespace` -- *all Chromium browsers support it natively*
- *Do NOT use browser. polyfill unless Firefox support is required*

Secrets: What Goes Where

Secret	Location	Never In
STRIPE_SECRET_KEY	Supabase Edge Function (Deno.env)	Extension bundle
STRIPE_WEBHOOK_SECRET	Supabase Edge Function (Deno.env)	Extension bundle
SUPABASE_SERVICE_ROLE_KEY	Supabase Edge Function (Deno.env)	Extension bundle
SUPABASE_URL	Extension (build-time env var, public)	--
SUPABASE_ANON_KEY	Extension (build-time env var, public)	--
Stripe publishable key (pk_live)	Extension (public, safe to expose)	--

Run: `supabase secrets set KEY=value`

Reference: <https://supabase.com/docs/guides/functions#secrets-and-environment-variables>